School of Informatics & Cybersecurity - Assessment Brief Coversheet

Assessment Information

Module Title:	**Network Security**
Module Code:	**CYSC H1001**
Assessment Title:	**Open Source Firewalls Assignment**
Assessment Type (e.g. Essay, Report, Presentation, Code submission):	**Report, Presentation & Demonstration**
Assessment Weighting (%):	**30% CA**
Submission Deadline:	**11.59pm 02/11/2025**
Late Submission Penalty:	**0% for late submissions**

Use of Generative AI

☑ The use of generative AI tools (e.g., ChatGPT, DALL·E, etc.) is NOT permitted for this assessment.

☐The use of generative AI tools IS permitted under the following conditions:

Conditions and limitations for use:

N/A – AI tools are NOT permitted for this assessment

Lecturer Name: Peter Alexander

Date: 06/10/2025

IPFire

Connectivity Test

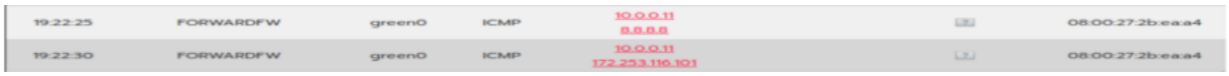
The objective of the connectivity test is to verify that th

e clients on the Green network can successfully access the internet through the RED zone through the IPfire firewall.

```
vboxuser@Xubuntu:~$ ping -c 4 google.com
PING google.com (172.253.116.100) 56(84) bytes of data.
64 bytes from dj-in-f100.1e100.net (172.253.116.100): icmp_seq=1 ttl=254 time=24.4 ms
64 bytes from dj-in-f100.1e100.net (172.253.116.100): icmp_seq=2 ttl=254 time=53.0 ms
64 bytes from dj-in-f100.1e100.net (172.253.116.100): icmp_seq=3 ttl=254 time=49.4 ms
64 bytes from dj-in-f100.1e100.net (172.253.116.100): icmp_seq=4 ttl=254 time=31.3 ms
```

```
vboxuser@Xubuntu:~$ ping -c 4 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=254 time=50.3 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=254 time=68.3 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=254 time=39.5 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=254 time=36.4 ms

--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3049ms
rtt min/avg/max/mdev = 36.390/48.619/68.260/12.458 ms
vboxuser@Xubuntu:~$
```

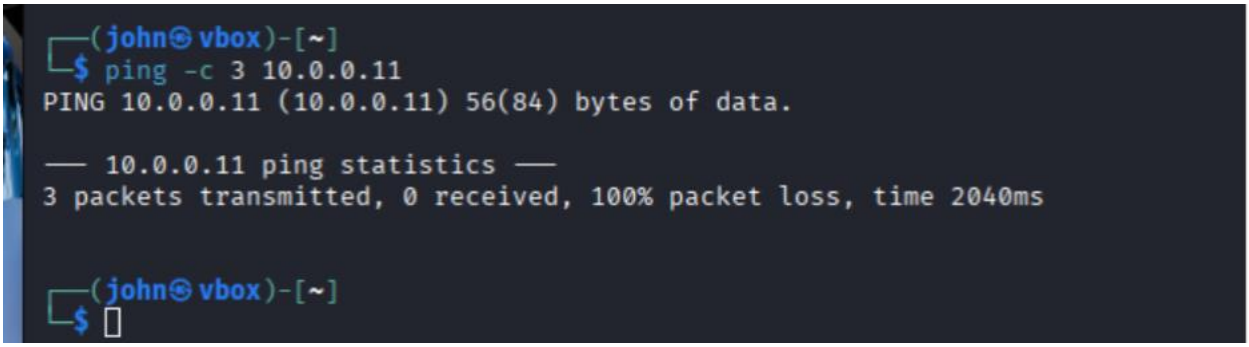


As we can see, all the pings were successful with no packet loss. The IPFire logs show ICMP entries from the Client in the green network (10.0.0.11) to the above addresses which confirm that traffic was routed and monitored correctly.

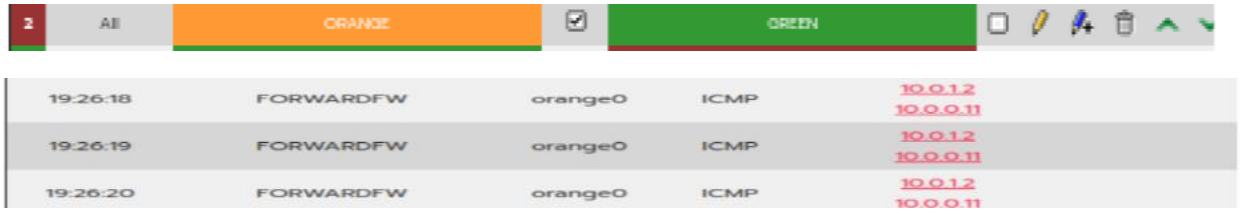
ICMP Block test

ICMP pinging is one of the first tools attackers use in order to scan for hosts that are alive or reachable on a network. By blocking ICMP between zones, this test will demonstrate how IPFire prevents attackers from mapping or scanning the internal LAN. It ensures that our green network hosts remain invisible to attackers.

I ran an ICMP block test to verify that the firewall blocks ICMP ping requests coming from the attacker machine on the ORANGE zone (10.0.1.2) to the internal LAN client in the green zone (10.0.0.11)



Bellow I have attached the rule that blocks the ICMP traffic.

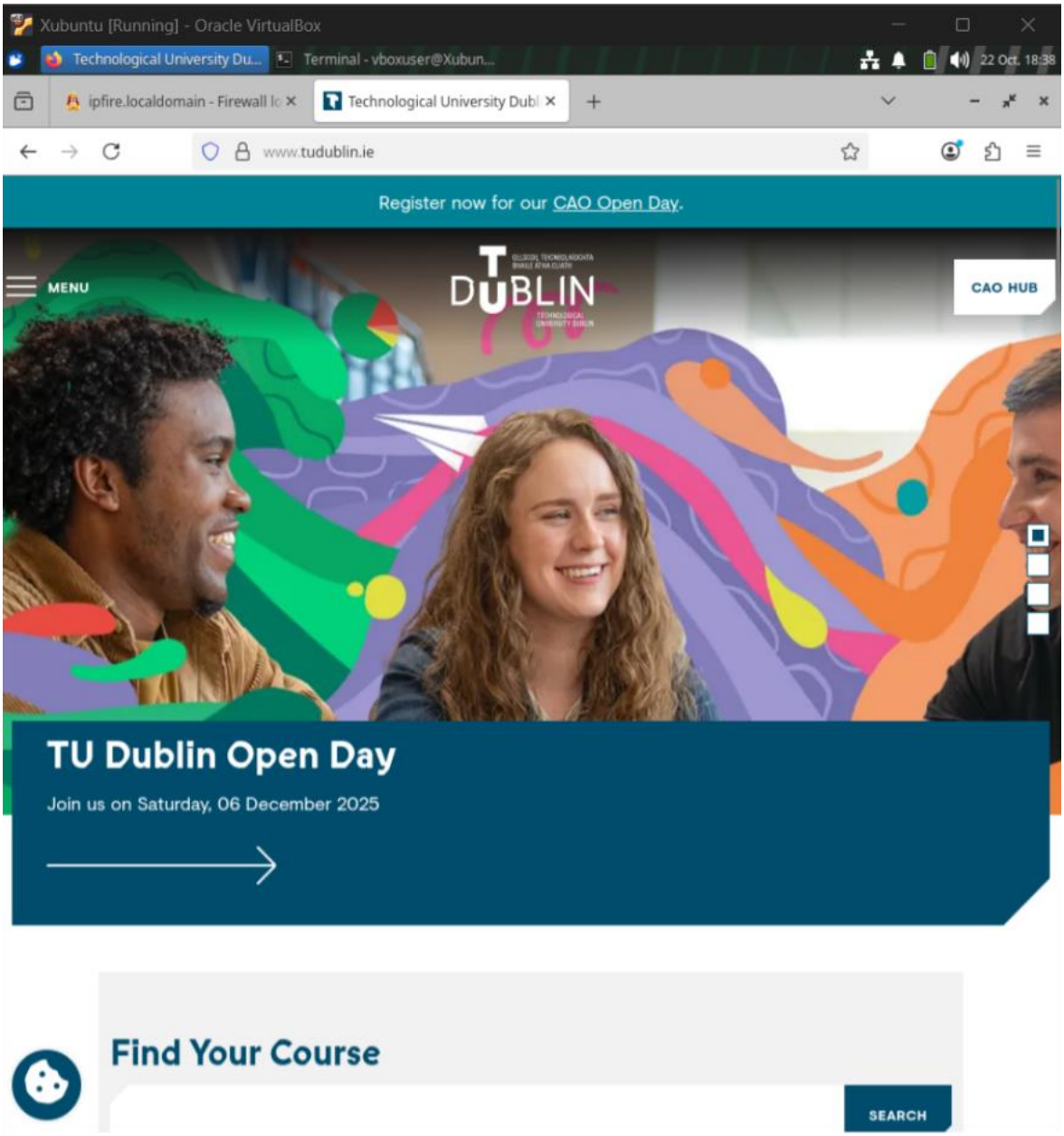


The test was successful as the attacker got the message that they received a 100% packet loss, and no packets reached the green network. The logs have been highlighted above to show the firewall detected the packets but didn't let them through, hence the rule we created.

HTTP/HTTPS from green to red test

I have ran a HTTP test in order to verify that IPFire allows outbound web traffic from internal green network to the red network while still protecting internal devices from incoming connections.

I used the client to open a browser and access the URL www.tudublin.ie



The test was successful as the client was able to reach the external web servers on the internet via the red interface, confirming that outbound traffic

(port 80 or 443) is permitted through the firewall.

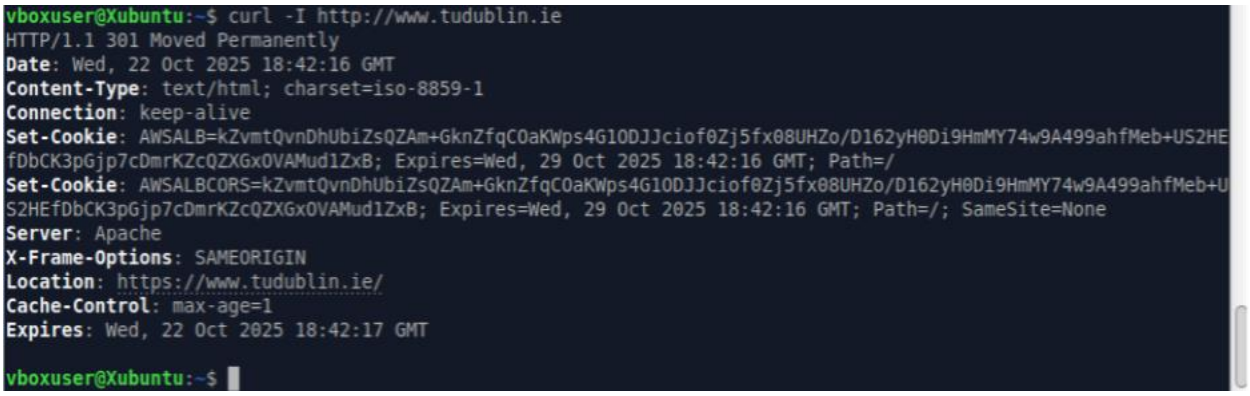
The web page loaded successfully, which lets us know that the firewall is correctly forwarding outbound traffic from green zone to the internet. This also let us know that the DNS and HTTP/HTTPS connections are functioning as expected and the client in the Green network has secure internet access, while inbound traffic from red interface remains blocked.

HTTP/HTTPS Traffic and Log verification Test.

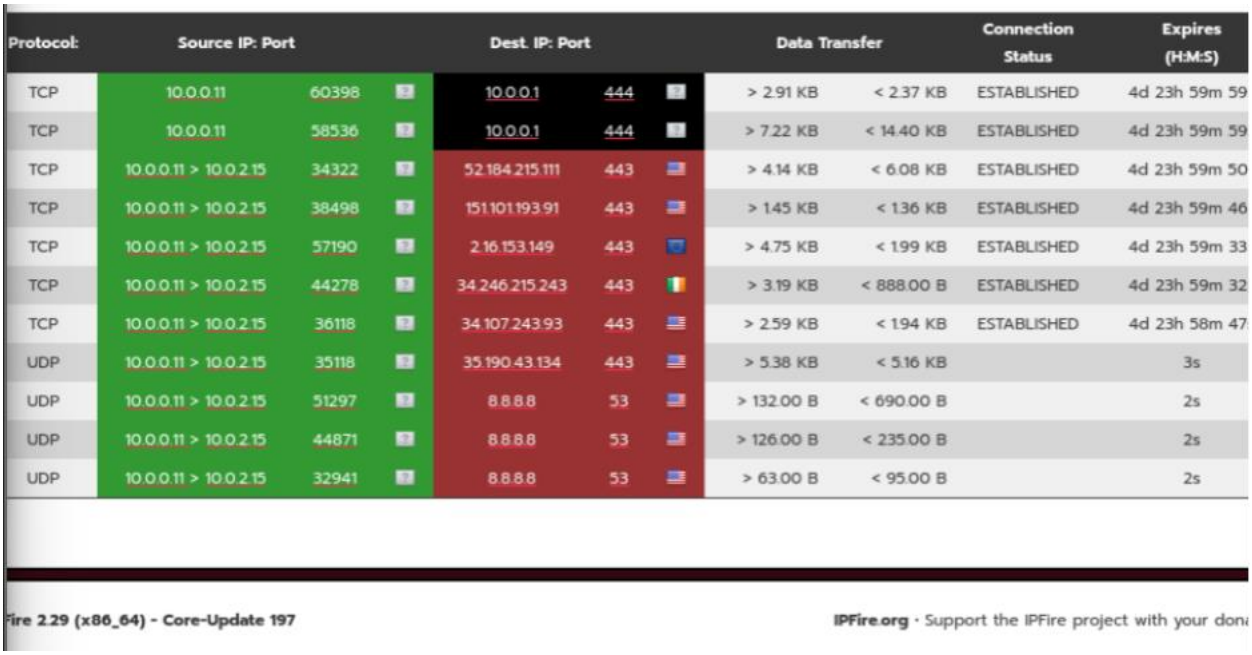
The purpose of a HTTPF and HTTPS Traffic and log verification test is to confirm that web traffic from the green network to the red network passes successfully Through the firewall and that IPfire accurately logs active connections and data transfers.

From the client host i ran the curl command to test web access at the terminal level.

The –I flag requests only HTTP header confirming a valid connection. The output returned a 301 Moved permanently message which means that the website successfully redirected from HTTP to HTTPS.



Then i went onto the firewall into the “Connections” tab under “Status” to observe the live network sessions.



Protocol:	Source IP: Port		Dest. IP: Port		Data Transfer		Connection Status	Expires (H:M:S)
TCP	10.0.0.11	60398	10.0.0.1	444	> 2.91 KB	< 2.37 KB	ESTABLISHED	4d 23h 59m 59
TCP	10.0.0.11	58536	10.0.0.1	444	> 7.22 KB	< 14.40 KB	ESTABLISHED	4d 23h 59m 59
TCP	10.0.0.11 > 10.0.2.15	34322	52.184.215.111	443	> 4.14 KB	< 6.08 KB	ESTABLISHED	4d 23h 59m 50
TCP	10.0.0.11 > 10.0.2.15	38498	151.101.193.91	443	> 1.45 KB	< 1.36 KB	ESTABLISHED	4d 23h 59m 46
TCP	10.0.0.11 > 10.0.2.15	57190	2.16.153.149	443	> 4.75 KB	< 1.99 KB	ESTABLISHED	4d 23h 59m 33
TCP	10.0.0.11 > 10.0.2.15	44278	34.246.215.243	443	> 3.19 KB	< 888.00 B	ESTABLISHED	4d 23h 59m 32
TCP	10.0.0.11 > 10.0.2.15	36118	34.107.243.93	443	> 2.59 KB	< 1.94 KB	ESTABLISHED	4d 23h 58m 47
UDP	10.0.0.11 > 10.0.2.15	35118	35.190.43.134	443	> 5.38 KB	< 5.16 KB		3s
UDP	10.0.0.11 > 10.0.2.15	51297	8.8.8.8	53	> 132.00 B	< 690.00 B		2s
UDP	10.0.0.11 > 10.0.2.15	44871	8.8.8.8	53	> 126.00 B	< 235.00 B		2s
UDP	10.0.0.11 > 10.0.2.15	32941	8.8.8.8	53	> 63.00 B	< 95.00 B		2s

When we log onto IPfire and check the status of Connections, we can see that our client IP made a HTTP request through IPfire. Here we can log all the requests made by our client. We can look at protocol and define it. TCP is a HTTPS request as in a UDP are DNS queries.

HTTP/HTTPS Log verification Test confirmed that web access from the green interface to the red is functioning correctly. IPFire firewall acurately tracked, recorded and displayed active traffic in real time.

This test is an essencial part of a firewall because it provides visibility into network behaviour and threat analysis.

DNS filtering and port 53 Blocking Test

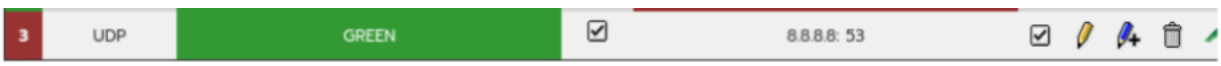
I have concluded a DNS filtering and Port 53 blocking test in order to verify that IPFire can effectively block unauthorised DNS request from internal clients and ensure that all DNS queries are routed through the firewalls own resolver tather than directly to external DNS servers such as 8.8.8.8.

This test will ensure network visibility, central control and prevention of DNS bypass, which are critical in enterprise or security-focused environments.

So to run this test I had to go on the firewall to create a rule to block all the UDP traffic from the green network that is destined for port 53 on the red interface.

I used port 53 because it is the default port for DNS protocol.

So the rule attached below ensures that clients cannot query external DNS servers directly and instead all DNS lookup must pass through IPfires built in DNS proxy where filtering and monitoring can take place.



On the Clients system i ran the command “nslookup www.tudublin.ie 8.8.8.8” to simulate a manual DNS request.

```
vboxuser@Xubuntu:~$ nslookup www.tudublin.ie 8.8.8.8
;; communications error to 8.8.8.8#53: timed out
;; communications error to 8.8.8.8#53: timed out
;; communications error to 8.8.8.8#53: timed out
;; no servers could be reached

vboxuser@Xubuntu:~$
```

Below we can see the firewall logs that confirm the UDP packets from the green network tried to go for the external DNS server on port 53 but got dropped imediately by the firewall.



20:18:14	FORWARDFW	green0	UDP	10.0.0.11 8.8.8.8	43759 53(DOMAIN)
20:18:19	FORWARDFW	green0	UDP	10.0.0.11 8.8.8.8	38347 53(DOMAIN)
20:18:24	FORWARDFW	green0	UDP	10.0.0.11 8.8.8.8	35223 53(DOMAIN)

The DNS blocking test confirmed that IPfire was able to control outbound DNS traffic, ensuring the clients cannot bypass the firewalls internal DNS service.

This test reinforces both network security and administrative oversight by forcing all DNS lookups through a single, monitored and trusted source.

DDoS attack

So, for the last test I simulated a real-life threat. DDoS attacks or also known as a Distributed Denial of service attack, are meant to overwhelm a network or system by flooding it with excessive volume of traffic or resource requests.

The Goal of a DDoS attack is to degrade availability and saturate network availability and bandwidth or overloading the CPU and memory so legitimate users cannot access services.

For this lab i used hping3 because it can generate different protocol mixes (TCP, UDP, ICMP) .

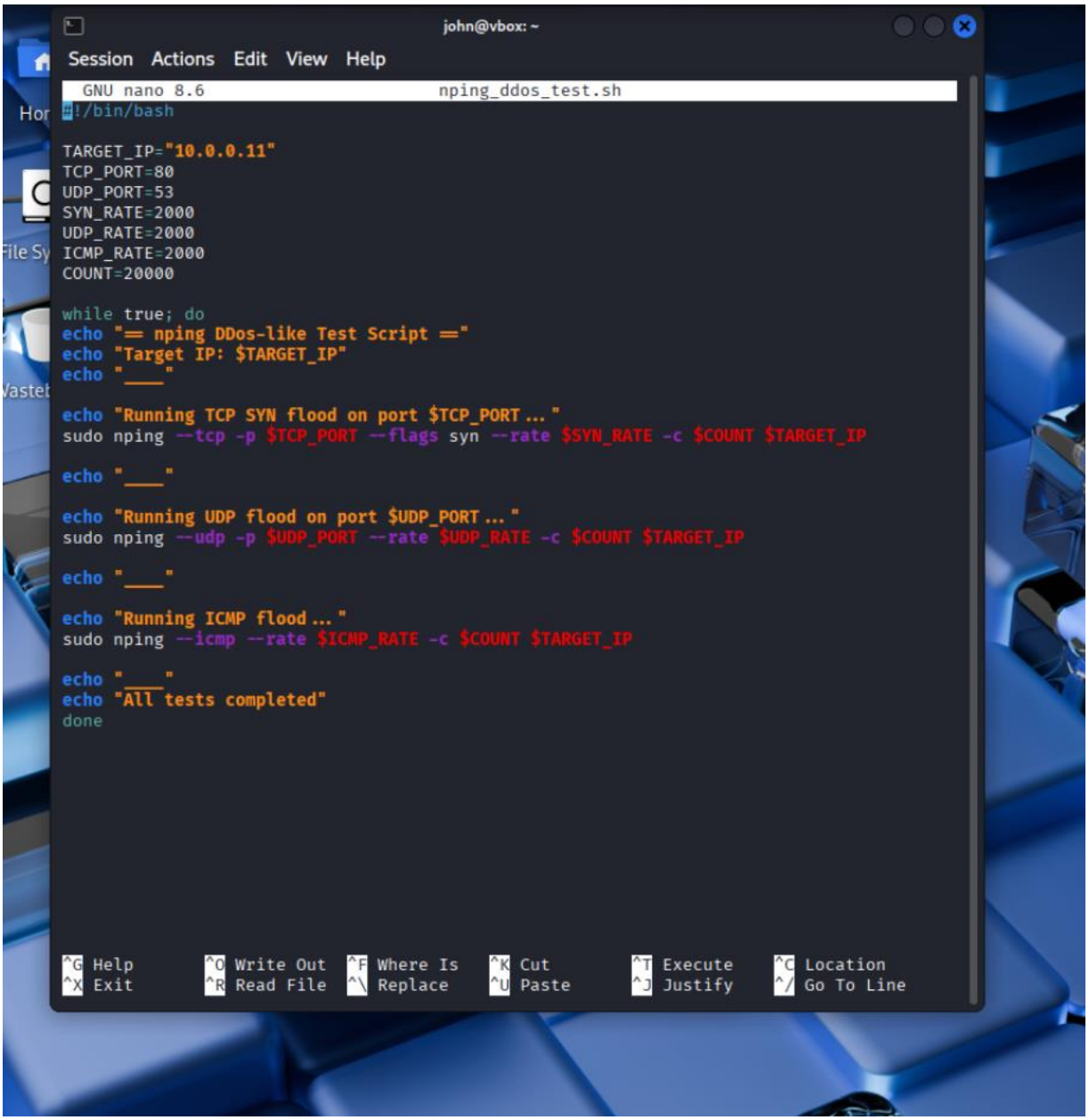
So i created a file on my attacker system (kali linux) and adited it by inserting the following commands for TCP SYN flooding, UDP flooding and ICMP flooding. The TCP port i used was 80 and port 53 for UDP. Very common ports used.

I have disabled any rules that block the connectivity from the orange network to the green network for the attack to take place.

When running the attack, using the web on the target system was impossible. Nothing crashed or gave errors but seemed like the internet was very slow. And just never responded.

After i cancelled the attack off the attacker system i enabled all the rules again.

Now i started the attack again and using the web services felt like nothing was going on. I logged all the packets and i can confirm the firewall was doing its job.


john@vbox: ~
Session Actions Edit View Help
GNU nano 8.6 nping_ddos_test.sh
#!/bin/bash
TARGET_IP="10.0.0.11"
TCP_PORT=80
UDP_PORT=53
SYN_RATE=2000
UDP_RATE=2000
ICMP_RATE=2000
COUNT=20000
while true; do
echo "== nping DDos-like Test Script =="
echo "Target IP: $TARGET_IP"
echo "____"
echo "Running TCP SYN flood on port $TCP_PORT ... "
sudo nping --tcp -p $TCP_PORT --flags syn --rate $SYN_RATE -c $COUNT $TARGET_IP
echo "____"
echo "Running UDP flood on port $UDP_PORT ... "
sudo nping --udp -p $UDP_PORT --rate $UDP_RATE -c $COUNT $TARGET_IP
echo "____"
echo "Running ICMP flood ... "
sudo nping --icmp --rate $ICMP_RATE -c $COUNT $TARGET_IP
echo "____"
echo "All tests completed"
done
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line

Here are some logs from the DDoS attack.



21:37:32	DROP_FORWARD	orange0	ICMP	10.0.1.2 10.0.0.11		08:00:27:d2:78:f4
21:37:32	DROP_FORWARD	orange0	ICMP	10.0.1.2 10.0.0.11		08:00:27:d2:78:f4
21:37:32	DROP_FORWARD	orange0	ICMP	10.0.1.2 10.0.0.11		08:00:27:d2:78:f4
21:37:32	DROP_FORWARD	orange0	ICMP	10.0.1.2 10.0.0.11		08:00:27:d2:78:f4
21:37:32	DROP_FORWARD	orange0	ICMP	10.0.1.2 10.0.0.11		08:00:27:d2:78:f4
21:37:32	DROP_FORWARD	orange0	ICMP	10.0.1.2 10.0.0.11		08:00:27:d2:78:f4
21:37:32	DROP_FORWARD	orange0	ICMP	10.0.1.2 10.0.0.11		08:00:27:d2:78:f4
21:37:32	DROP_FORWARD	orange0	ICMP	10.0.1.2 10.0.0.11		08:00:27:d2:78:f4
21:37:32	DROP_FORWARD	orange0	ICMP	10.0.1.2 10.0.0.11		08:00:27:d2:78:f4

First Older Newer Last

The logs let us know a lot of valuable information such as the source ip like the attacker machine from the orange network, the destination ip or victim, the protocol used (ICMP,TCP or UDP) depending on which type of packet flood was used, timestamps and status to show the packets were intercepted and dropped.

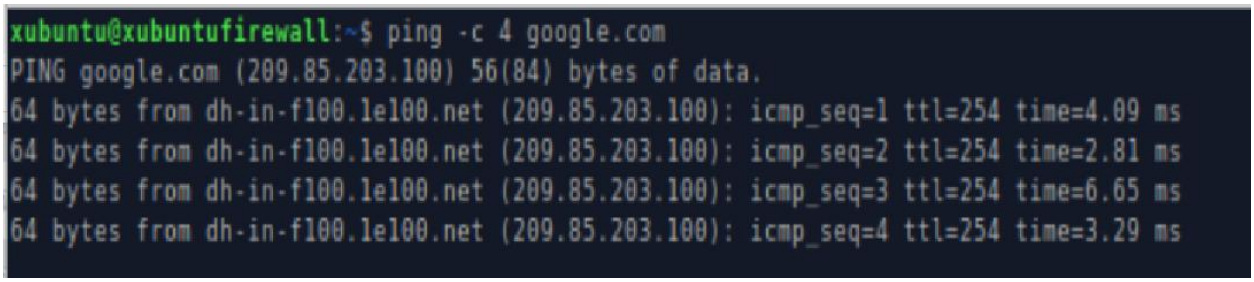
The large number of identical log entries in such a short time frame indicates unusual and excessive traffic, which is a clear sign of a denial of service pattern. Ipfire's logs confirm that the firewall did not allow this flood to pass through to the victim system, protecting the internal network from being overwhelmed.

Conclusion

So IPFire proved to be an effective tool for securing and managing network traffic. Throughout my tests, it successfully controlled the traffic and comunication between the Red, Gree, and Orange networks. It allowed normal access from the internal LAN to the internet while blocking unwanted or harmful traffic such as pings from the attacker and DNS requests on port 53. During the DDoS simulation, IPFire detected and logged the attack without allowing it to reach the target. Overall, the firewall provided strong protection, clear visibility through logs and reliable performance in maintaining network safety stability.

Test from IPCop firewall:

Test NO:1 Connectivity



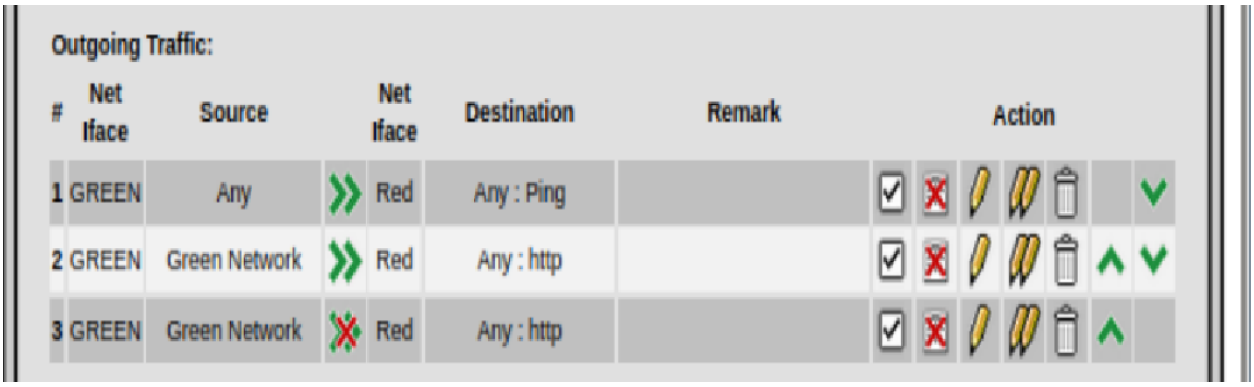
In the screenshot above I ran the command ping –c 4 google.com as you can see from the screenshot from my Xubuntu virtual machine (VM). Which is the client. The purpose of this test was to make sure that the client had full network connectivity through the IPCop firewall before doing any rule or attack. As shown in the picture, all four packets were sent and received successfully from Google servers. In the picture, you can see how long the response times are. This confirms that the connection is fast and stale. So, this result proves that the Xubuntu machine is correctly connected to the IPCop firewall and that the firewall allows outgoing ICMP traffic to the Interne. This step was important because it confirmed that my network setup was working properly before applying any firewall restrictions.it also helped that my DNS and routing configuration were correct which meaning that my Xubuntu client could reach without any issues. Before doing any rule testing, I wanted to be sure that all the connections between my VMs were set up correctly. By using the ping command, I could see both the connectivity and the response time between my system and the external network. It was also useful to confirm that my IPCop firewall was not blocking basic outgoing requests. This type of test is usually the first step in any network troubleshooting because it shows that the firewall, NAT, and DNS are configured correctly. If this test failed, it would mean there was an issue either with the green interface, routing table, or the DNS settings. But since it worked fine, it gave me

confidence to move forward with the next firewall rules and access control tests. It also showed that my network was stable and that the packets were going through the correct interface.

Test NO:2 Access Control Block HTTP:

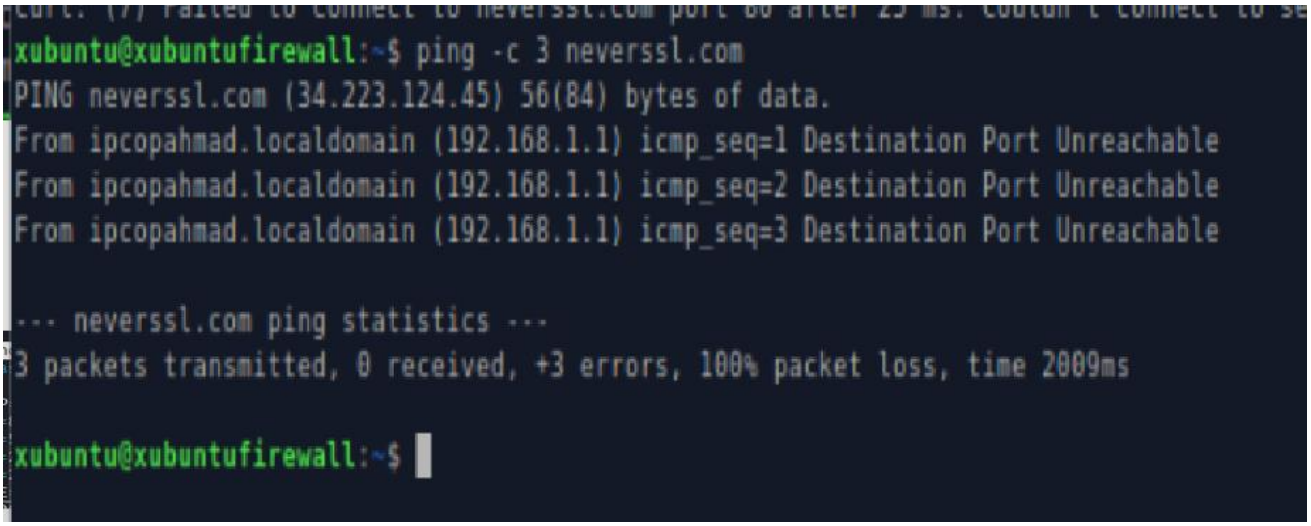
The second test I did was about blocking HTTP port 80. I did this test because I was trying to test the IPCop to access control functionality by creating a rule to block outbound HTTTP port 80 from green network. After I applied for the rule, I tried to access an HTTP site from the Xubuntu client using curl. The request failed as you can see in the screenshot. This shows that the firewall actively stopped the traffic. The IPCop rule table in the screenshot you can the rule where I add it. This test clearly shows that my IPCop firewall is working as expected. It successfully blocked access to the websites using HTTP, which means the rule I set up was correctly applied. This test was important because it showed that the IPCop firewall can control what kind of traffic leaves the network. Port 80 is used for normal website browsing that is not encrypted, so blocking it helps protect the system from unsafe or unwanted sites. I added the rule by going into the IPCop web interface, selecting Firewall → Rules, and creating a new one to deny all traffic from the green zone to any destination on port 80. After saving and applying, I tested again using curl and a web browser. Both failed to connect to normal HTTP websites, which confirmed that the rule worked perfectly. This is a common rule used by network administrators to make sure users only use HTTPS connections for secure browsing. It also helped me understand how

different protocols use specific ports, and how easy it is to control them using IPCop's graphical interface



Outgoing Traffic:

#	Net Iface	Source		Net Iface	Destination	Remark	Action
1	GREEN	Any	»	Red	Any : Ping		
2	GREEN	Green Network	»	Red	Any : http		
3	GREEN	Green Network	✗	Red	Any : http		



```
xubuntu@xubuntufirewall:~$ ping -c 3 neverssl.com
PING neverssl.com (34.223.124.45) 56(84) bytes of data.
From ipcopahmad.localdomain (192.168.1.1) icmp_seq=1 Destination Port Unreachable
From ipcopahmad.localdomain (192.168.1.1) icmp_seq=2 Destination Port Unreachable
From ipcopahmad.localdomain (192.168.1.1) icmp_seq=3 Destination Port Unreachable

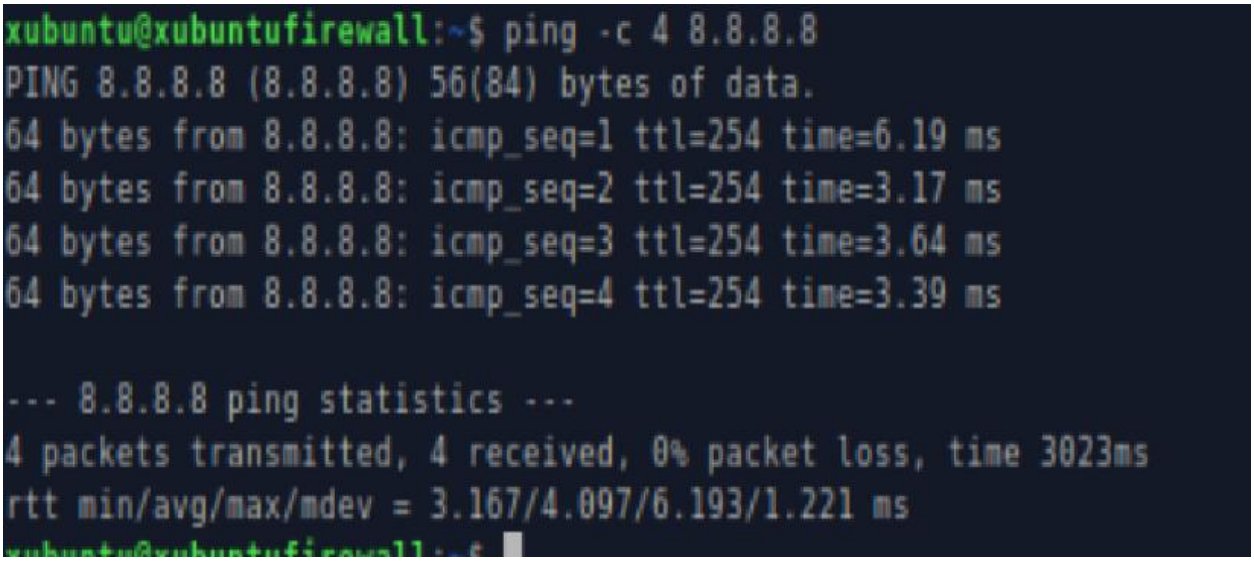
--- neverssl.com ping statistics ---
3 packets transmitted, 0 received, +3 errors, 100% packet loss, time 2009ms

xubuntu@xubuntufirewall:~$
```



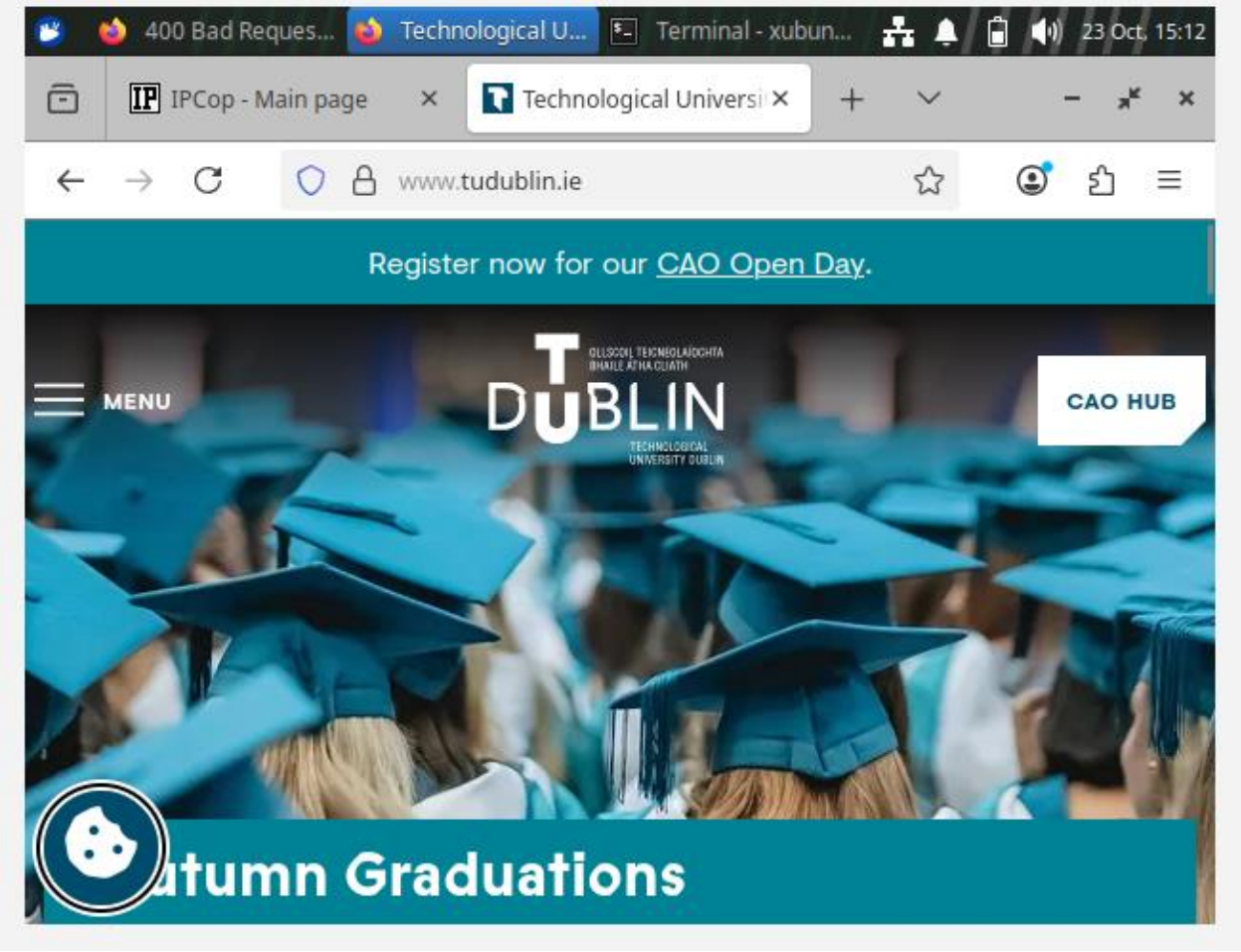
```
xubuntu@xubuntufirewall:~$ curl -I --max-time 10 http://neverssl.com
curl: (7) Failed to connect to neverssl.com port 80 after 25 ms: Couldn't connect to server
xubuntu@xubuntufirewall:~$
```

This Screenshot shows the test that I used the command ping –c 4 8.8.8.8 as you can see in the screenshot to check if my Xubuntu client could reach the Internet through the IPCop firewall. All four packets were received with the lest response times, which tell us that the connection is working. I did this to make sure the network was set up correctly before running the other firewall tests.



Test: Web Access:

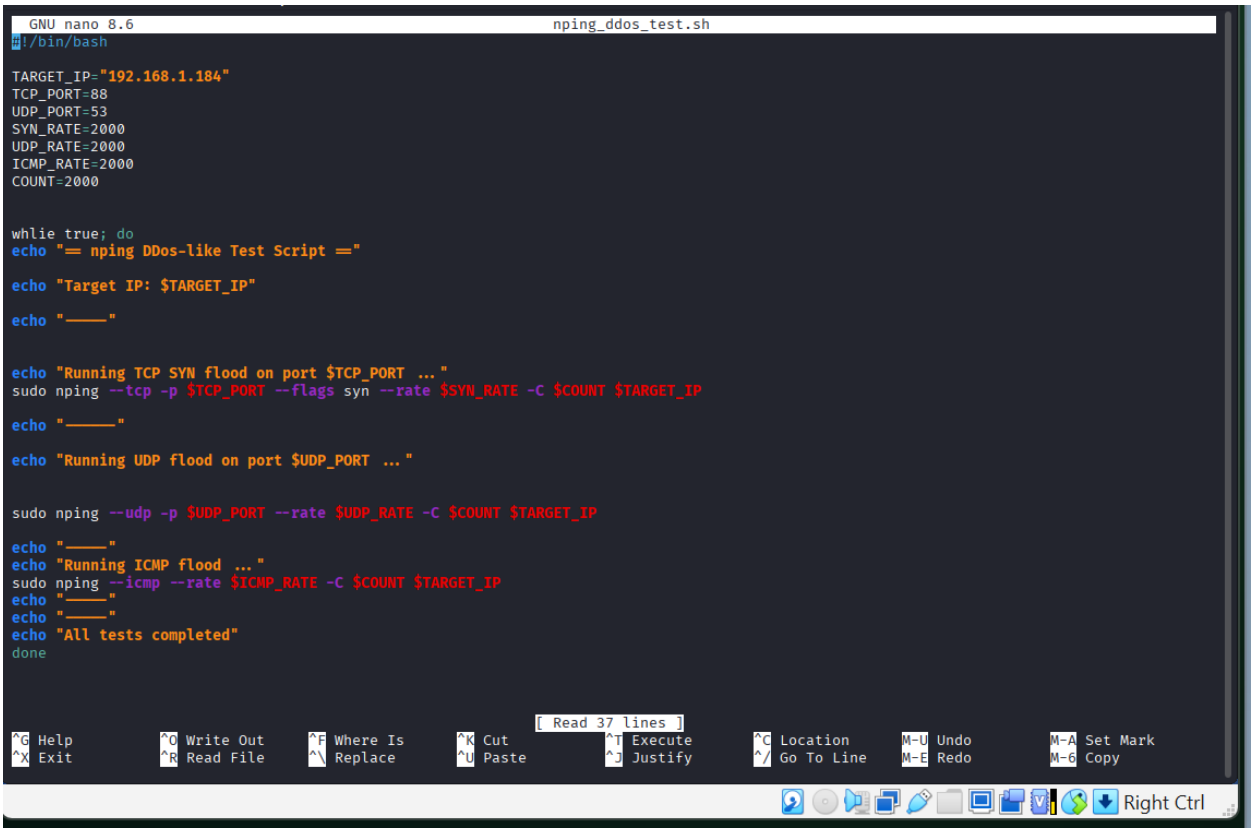
This Screenshot is the TU Dublin website. I did this test to check if the Xubuntu client can access websites through the IPCop firewall. I used the TU Dublin website as an example to confirm that HTTP and HTTPS traffic were allowed. The page loaded successfully, which shows the firewall rules for web browsing were correctly configured, and the Internet is connected and working. This test also proves that the firewall is functioning as expected, allowing safe and secure access to websites while still protecting the network from unwanted traffic.



Test: DDoS Attack:

In this test, I did the DDoS attack to see how my IPCop reacts under heavy traffic. I used a simple nping script on my Kali which is my attacker to send TCP,UDP,and ICMP packets to the Xubuntu target. The goal was to flood the target with requests and check how the firewall handled it. This test was useful for a few reasons. First is to see whether IPCop would detect and log the sudden increase in traffic and the second to check whether it would block or drop the malicious packets before they overwhelm the target.While the script was running, I watched target htop and if top to see the CPU and bandwidth spikes. and refreshed the IPCop logs to capture blocked entries.

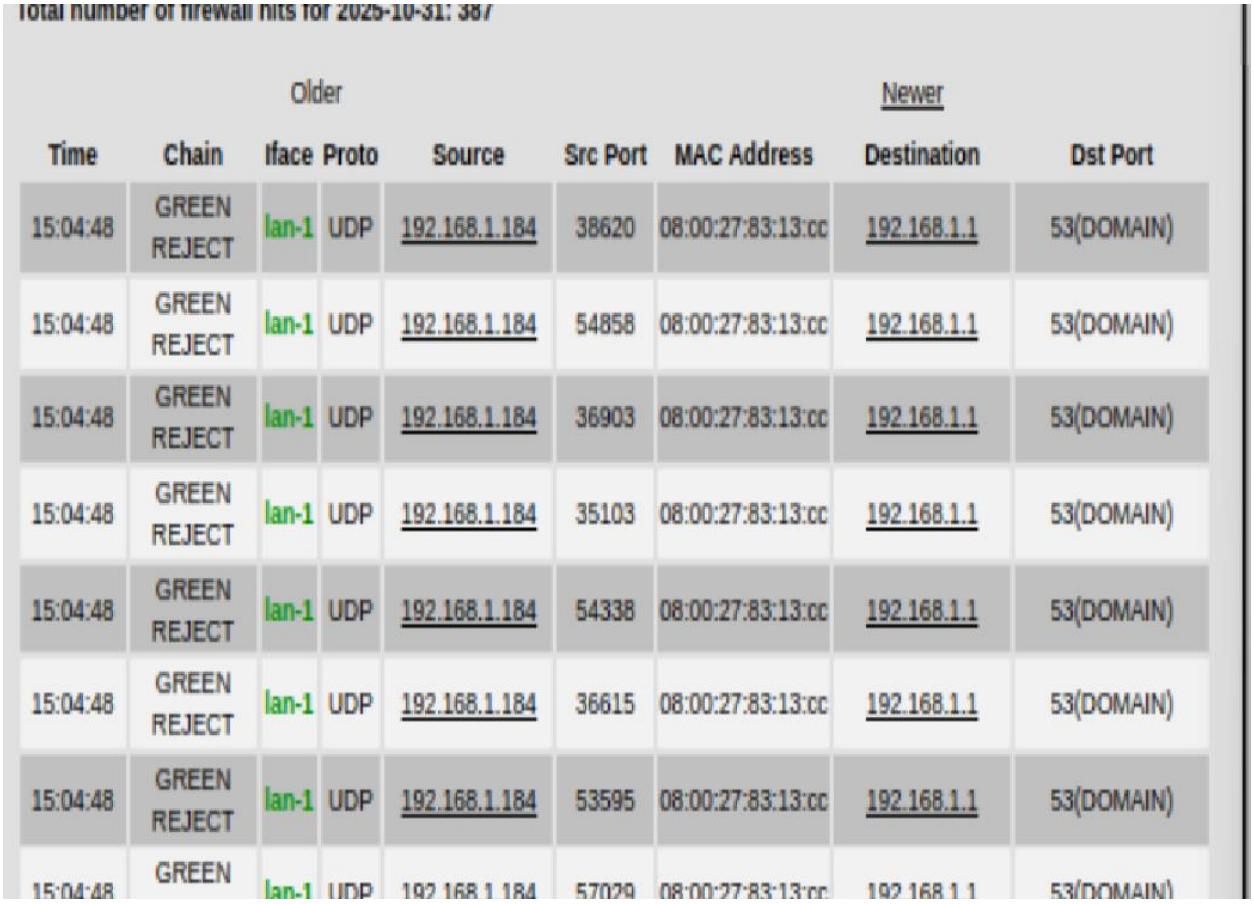
The first screenshot shows the attack script I used where I set packet rates and target IP.



The second screenshot (top) was taken from the Xubuntu client, showing the CPU and memory usage going up during the attack. This tells us that the target was under load.


0[|||||| 18.6%] Tasks: 116, 566 thr, 86 kthr; 2 run
1[|||||||||| 31.3%] Load average: 1.49 0.73 0.45
Mem[||||||||||||||||||1.24G/1.92G] Uptime: 06:34:47
Swp[0K/0K]
Main I/O
PID USER PRI NI VIRT RES SHR S CPU%▽MEM% TIME+ Command
1707 xubuntu 20 0 449M 33236 24636 S 0.6 1.6 0:02.24 /usr/lib
2286 xubuntu 20 0 3148M 373M 88060 S 0.6 19.0 2:04.46 /snap/fi
2638 xubuntu 20 0 2496M 133M 28316 S 0.6 6.8 0:01.46 /snap/fi
8760 xubuntu 20 0 299M 6220 5708 S 0.6 0.3 0:01.63 /usr/lib
1 root 20 0 22780 12880 8400 S 0.0 0.6 0:13.07 /sbin/in
323 root 20 0 30592 8212 4628 S 0.0 0.4 0:01.51 /usr/lib
423 systemd-re 20 0 21716 11616 9312 S 0.0 0.6 0:02.20 /usr/lib
426 systemd-ti 20 0 91048 7564 6668 S 0.0 0.4 0:00.38 /usr/lib
492 systemd-ti 20 0 91048 7564 6668 S 0.0 0.4 0:00.00 /usr/lib
664 root 20 0 303M 7572 6804 S 0.0 0.4 0:00.18 /usr/lib
673 avahi 20 0 8608 4224 3840 S 0.0 0.2 0:00.32 avahi-da
676 root 20 0 6824 2640 2512 S 0.0 0.1 0:00.16 /usr/sbi
677 messagebus 20 0 11096 6596 4420 S 0.0 0.3 0:15.04 @dbus-da
689 root 20 0 303M 7572 6804 S 0.0 0.4 0:00.00 /usr/lib
F1Help F2Setup F3SearchF4FilterF5Tree F6SortByF7Nice -F8Nice +F9Kill F10

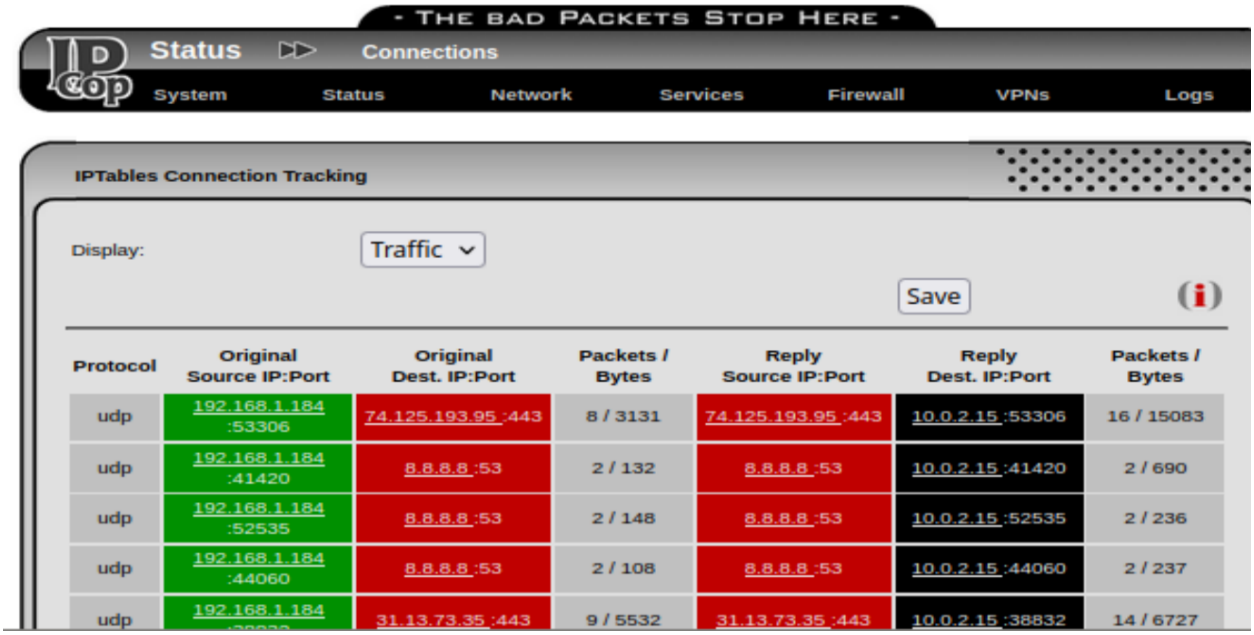
So overall, the purpose of this test is to show how the real attack could try to overload a network and how IPCop can detect and block such traffic. When I checked IPCop logs after the test, I saw many block packets. Which told us the firewall was doing its job. Once the test was stopped, the network whet back to normal. This screenshot shows IPCop firewall log during the DDoS test. This shows the multiple UDP packets from the Xubuntu client to the firewall on port 53. These packets, as shown as Reject, have told us the firewall detected and blocked them. This shows that the firewall was working correctly and blocked unwanted traffic during this test.



Total number of firewall hits for 2025-10-31: 387

		Older					Newer	
Time	Chain	Iface	Proto	Source	Src Port	MAC Address	Destination	Dst Port
15:04:48	GREEN REJECT	lan-1	UDP	192.168.1.184	38620	08:00:27:83:13:cc	192.168.1.1	53(DOMAIN)
15:04:48	GREEN REJECT	lan-1	UDP	192.168.1.184	54858	08:00:27:83:13:cc	192.168.1.1	53(DOMAIN)
15:04:48	GREEN REJECT	lan-1	UDP	192.168.1.184	36903	08:00:27:83:13:cc	192.168.1.1	53(DOMAIN)
15:04:48	GREEN REJECT	lan-1	UDP	192.168.1.184	35103	08:00:27:83:13:cc	192.168.1.1	53(DOMAIN)
15:04:48	GREEN REJECT	lan-1	UDP	192.168.1.184	54338	08:00:27:83:13:cc	192.168.1.1	53(DOMAIN)
15:04:48	GREEN REJECT	lan-1	UDP	192.168.1.184	36615	08:00:27:83:13:cc	192.168.1.1	53(DOMAIN)
15:04:48	GREEN REJECT	lan-1	UDP	192.168.1.184	53595	08:00:27:83:13:cc	192.168.1.1	53(DOMAIN)
15:04:48	GREEN	lan-1	UDP	192.168.1.184	57029	08:00:27:83:13:cc	192.168.1.1	53(DOMAIN)

This screenshot the connection tracking page on my IPCop firewall.it lists all the active connection tracking page on my IPCop firewall.it lists all the active connections going through the firewall at the time. You can see my Xubuntu client (192.168.1.184) making UDP connection to public IPs like 8.8.8.8 and 74.125.193.95, which are Google servers. This means the client was sending DNS and web requests through the IPCop, and the firewall kept track of every packet. I took this picture to show that IPCop is monitoring live network traffic correctly. And all those connections are being logged as they pass through the firewall.



Protocol	Original Source IP:Port	Original Dest. IP:Port	Packets / Bytes	Reply Source IP:Port	Reply Dest. IP:Port	Packets / Bytes
udp	192.168.1.184 :53306	74.125.193.95 :443	8 / 3131	74.125.193.95 :443	10.0.2.15 :53306	16 / 15083
udp	192.168.1.184 :41420	8.8.8.8 :53	2 / 132	8.8.8.8 :53	10.0.2.15 :41420	2 / 690
udp	192.168.1.184 :52535	8.8.8.8 :53	2 / 148	8.8.8.8 :53	10.0.2.15 :52535	2 / 236
udp	192.168.1.184 :44060	8.8.8.8 :53	2 / 108	8.8.8.8 :53	10.0.2.15 :44060	2 / 237
udp	192.168.1.184	31.13.73.35 :443	9 / 5532	31.13.73.35 :443	10.0.2.15 :38832	14 / 6727

Conclusion:

From this DDos test, I learned how the IPCop firewall reacts when the network faces a large amount of traffic. The firewall successfully defected and blocked the unwanted packets while keeping the system stable.it also logged the attack attempts, which is helpful for monitoring and analysis. This test shows that IPCop can protect the internal network (Xubuntu) from heavy or suspicious traffic coming from external attackers (Kali). Overall, it proved the firewall's strength and reliability under pressure. This test was important because it showed that the IPCop firewall can control what kind of traffic leaves the network. Port 80 is used for normal website browsing that is not encrypted, so blocking it helps protect the system from unsafe or unwanted sites. I added the rule by going into the IPCop web interface, selecting Firewall → Rules, and creating a new one to deny all traffic from the green zone to any destination on port 80. After saving and applying, I tested again using curl and a web browser. Both failed to connect to normal HTTP websites, which confirmed that the rule worked perfectly. This is a common rule used by network administrators to make sure users only use HTTPS connections for secure browsing. It also helped me understand how different protocols use specific ports, and how easy it is to control them using IPCop’s graphical interface.